Optimization of Matrix Multiplication and Sparse Formats

Individual Assignment 2 — Report

Carlos Montesdeoca Vega

repository: `https://github.com/CarlosMontesdeoca21/BasicMatrixMultiplication`

October 14, 2026

Abstract

This report presents a comprehensive evaluation of optimized techniques for matrix multiplication, covering cache-aware optimizations for dense matrices and sparse matrix strategies using the Compressed Sparse Row (CSR) format. We implement and benchmark a classical baseline, a loop-reordered optimization, a cache-blocked approach, and a sparse CSR implementation in Java. Performance is evaluated in terms of execution time, memory usage, and sensitivity to sparsity. Results demonstrate that loop reordering drastically improves cache locality for dense matrices, while sparse methods yield orders-of-magnitude improvements for matrices with high data scarcity.

1 Introduction

Matrix multiplication is a fundamental operation in scientific computing and data analytics. Its classical algorithm has cubic time complexity ($\mathcal{O}(n^3)$) and poor cache behavior, making it computationally expensive.

This project investigates two core optimization strategies:

- **Part A (Dense Optimizations):** Evaluation of a classical baseline against loop reordering and cache-blocking across matrix sizes $n \in \{10, 100, 250\}$.
- **Part B (Sparse Formats):** Comparison of dense multiplication against the Compressed Sparse Row (CSR) format for $n = 250$ across multiple sparsity levels.

All experiments were executed in Java, ensuring the isolation of the JIT compiler through warm-up iterations prior to the official measurements.

2 Methods

2.1 Dense Cache Optimizations

To evaluate the impact on the cache memory hierarchy (L1/L2), we implemented three dense approaches to compute the matrix product $C = A \times B$, where $A, B \in \mathbb{R}^{n \times n}$:

- **Baseline (Naive):** The traditional $i - j - k$ nested loop. It directly implements the standard algebraic definition:

$$C_{i,j} = \sum_{k=0}^{n-1} A_{i,k} B_{k,j} \tag{1}$$

  This approach suffers from poor spatial locality as it iterates through the columns of B, causing non-contiguous memory accesses in row-major languages like Java.

- **Opt1 (Loop Reordering):** Reordered to an $i - k - j$ loop structure. While mathematically identical to the baseline, this algorithm computes partial sums by iterating over the rows of B and C in the innermost loop. This guarantees contiguous stride-1 memory access, maximizing spatial locality.

- **Opt2 (Tiling/Blocking):** Matrices are partitioned into sub-matrices (tiles) of size $b \times b$ (where $b = 32$ in our experiments). The multiplication is performed at the block level rather than the scalar level:

$$C_{I,J} = C_{I,J} + \sum_{K} A_{I,K} B_{K,J} \tag{2}$$

  Where I, J, K represent the sub-blocks. This improves temporal locality by keeping the active blocks entirely within the fast L1/L2 cache during computation.

2.2 Sparse Matrix Operations (CSR)

Sparse matrices store only the non-zero values (NNZ) to save memory and avoid redundant $0 \times x$ operations. We implemented the Compressed Sparse Row (CSR) format, which compresses a matrix into three one-dimensional arrays:

- `values`: Stores the actual non-zero floating-point numbers.
- `col_indices`: Stores the respective column index for each value.
- `row_ptr`: Stores the starting index in the `values` array for each row.

Matrices of size $n = 250$ were evaluated across multiple sparsity levels: 50%, 75%, 90%, 95%, and 99% (where the "density" or non-zero values corresponds to 50%, 25%, 10%, 5%, and 1% respectively).

The paired logical inputs were generated through a custom Python script using a uniform random distribution (seed 42 for matrix A and seed 43 for matrix B). The conversion costs from dense to CSR format have been excluded from the Java time measurements to strictly evaluate the computational throughput of the multiplication itself.

3 Results

3.1 Part A: Dense Matrix Performance

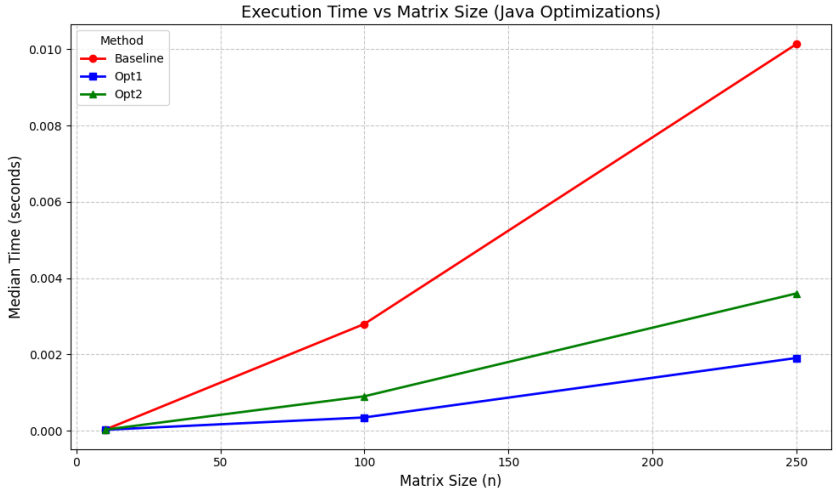


Figure 1: Execution Time vs. Matrix Size.

Table 1: Performance comparison of dense methods ($n = 250$)

Method	**Median Time (s)**	**Speedup vs Baseline**
Baseline ($i - j - k$)	0.0101	1.0×
Opt1 ($i - k - j$)	**0.0019**	**5.3**×
Opt2 (Tiling 32×32)	0.0036	2.8×

As shown in Figure 1 and Table 1, for size $n = 250$, the baseline method requires approximately 0.010s. Opt1 is over 5 times faster (0.0019s), and Opt2 yields 0.0036s.

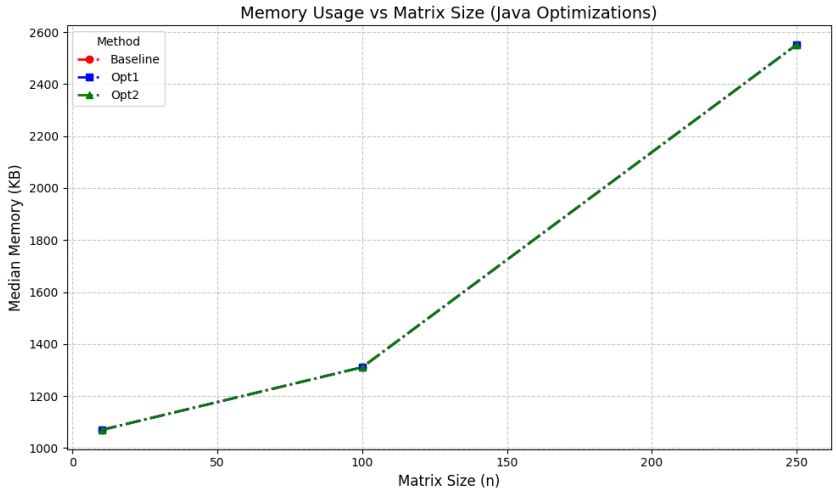


Figure 2: RAM Memory Consumption vs. Matrix Size.

Figure 2 confirms that all three methods consume exactly the same memory regardless of the execution time.

3.2 Part B: Sparse Matrix Performance

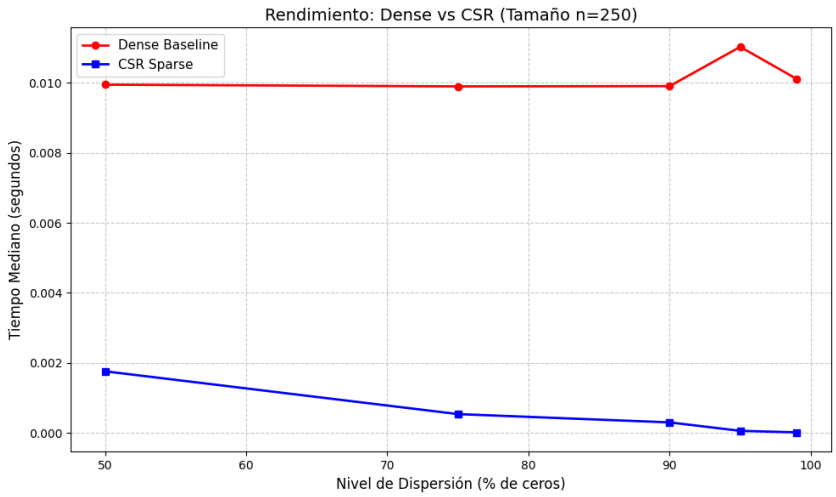


Figure 3: Crossover point: Dense Method vs CSR Format ($n = 250$).

In Figure 3, the dense algorithm maintains a constant execution time across all sparsity levels. The CSR format becomes faster starting from 50% sparsity, and its execution time approaches zero at 99% sparsity.

4 Discussion

Dense Multiplication: The severe performance bottleneck in the Baseline method is caused by continuous cache misses when reading matrix B by columns (*cache thrashing*). Loop reordering (Opt1) successfully exploits spatial locality (sequential memory access) without extra memory overhead. Tiling (Opt2) also improves locality but introduces management overhead for the nested blocks, making it slightly slower than Opt1 at this specific scale.

Sparse Multiplication: The CSR format imposes indexing latencies (*pointer chasing*). However, skipping the iterations over zeros easily compensates for this overhead past the 50% sparsity mark. The *fill-in* phenomenon was also observed: even though the input matrices have few non-zero values, the resulting product matrix C tends to become drastically denser.

5 Limitations and Future Work

- **Hardware and JVM Dependency:** The crossover point for sparse matrices and the optimal tile size for dense matrices are heavily dependent on the specific CPU L1/L2 cache sizes and Java's JIT compiler optimizations.

- **Sequential Execution:** This project evaluates purely sequential implementations. Future work could include multithreading or GPU acceleration (e.g., using CUDA/cuSPARSE).

6 Conclusion

It has been empirically demonstrated that a simple loop reordering (*i-k-j*) can quintuple the performance of dense matrix multiplication by isolating the positive impact of spatial locality, without incurring any memory costs.

Furthermore, the effects of sparse storage prove that traditional dense structures are inefficient for data scarcity. However, **it cannot be stated that a specific percentage of zeros determines the advantage in all universal cases**. The crossover point where CSR indexing becomes profitable depends critically on the matrix size (n), the hardware architecture, and the conversion costs to the compressed format.